

41. Develop a simple AI assistant capable of answering questions using external documents.

Code:

```
from sentence_transformers import SentenceTransformer
from transformers import pipeline
import faiss
import numpy as np

# =====
# SIMPLE AI DOCUMENT ASSISTANT
# =====

print("=====")
print("    DOCUMENT-BASED AI ASSISTANT")
print("=====")

# =====
# 1. EXTERNAL DOCUMENT
# =====

document = """
Artificial Intelligence is a branch of computer science
that focuses on creating intelligent systems.

Machine Learning is a subset of Artificial Intelligence.
It allows computers to learn patterns from data and
make predictions without being explicitly programmed.
```

Deep Learning is a type of Machine Learning that uses artificial neural networks with multiple layers.

Natural Language Processing helps computers understand, process and generate human language.

Computer Vision enables computers to understand and analyze images and videos.

Reinforcement Learning is a type of machine learning where an agent learns by receiving rewards and penalties.

"""

```
print("\nExternal document loaded successfully.")
```

```
# =====
```

```
# 2. SPLIT DOCUMENT INTO CHUNKS
```

```
# =====
```

```
chunks = [
```

```
    chunk.strip()
```

```
    for chunk in document.split("\n\n")
```

```
    if chunk.strip()
```

```
]
```

```
print("Number of document chunks:", len(chunks))
```

```
# =====  
# 3. LOAD EMBEDDING MODEL  
# =====
```

```
print("\nLoading embedding model...")
```

```
embedding_model = SentenceTransformer(  
    "all-MiniLM-L6-v2"  
)
```

```
print("Embedding model loaded.")
```

```
# =====  
# 4. CREATE DOCUMENT EMBEDDINGS  
# =====
```

```
print("\nCreating document embeddings...")
```

```
document_embeddings = embedding_model.encode(  
    chunks  
)
```

```
document_embeddings = np.array(  
    document_embeddings  
)  
.astype("float32")
```

```
print("Embeddings created successfully.")
```

```
# =====  
# 5. CREATE VECTOR DATABASE  
# =====
```

```
print("\nCreating FAISS vector database...")
```

```
dimension = document_embeddings.shape[1]
```

```
index = faiss.IndexFlatL2(dimension)
```

```
index.add(document_embeddings)
```

```
print("Vector database created.")
```

```
print("Documents stored:", index.ntotal)
```

```
# =====  
# 6. LOAD LANGUAGE MODEL  
# =====
```

```
print("\nLoading AI language model...")
```

```
llm = pipeline(  
    "text2text-generation",  
    model="google/flan-t5-small"  
)
```

```
print("Language model loaded.")
```

```
# =====
```

```
# 7. START AI ASSISTANT
```

```
# =====
```

```
print("\n=====")
```

```
print("      AI ASSISTANT READY")
```

```
print("=====")
```

```
print("Ask questions about the external document.")
```

```
print("Type 'exit' to stop.")
```

```
while True:
```

```
    # -----
```

```
    # Get question
```

```
    # -----
```

```
    question = input("\nYou: ")
```

```
    # -----
```

```
    # Exit
```

```
    # -----
```

```
    if question.lower() == "exit":
```

```
print("\nAssistant: Goodbye!")  
break
```

```
# =====
```

```
# 8. CREATE QUERY EMBEDDING
```

```
# =====
```

```
query_embedding = embedding_model.encode(  
    [question]  
)
```

```
query_embedding = np.array(  
    query_embedding  
)  
.astype("float32")
```

```
# =====
```

```
# 9. SEARCH VECTOR DATABASE
```

```
# =====
```

```
k = 2
```

```
distances, indices = index.search(  
    query_embedding,  
    k  
)
```

```
# =====  
# 10. RETRIEVE RELEVANT DOCUMENTS  
# =====
```

```
context = ""
```

```
for i in indices[0]:
```

```
    context += chunks[i] + "\n"
```

```
# =====  
# 11. CREATE PROMPT  
# =====
```

```
prompt = ""
```

You are a document-based AI assistant.

Answer the user's question using only the  
information in the provided document context.

If the answer is not available, say:

"I could not find the answer in the document."

Document Context:

```
{
```

Question:

```
}
```

Answer:

```
"".format(  
    context,  
    question  
)
```

```
# =====  
# 12. GENERATE ANSWER  
# =====
```

```
result = llm(  
    prompt,  
    max_new_tokens=100  
)
```

```
answer = result[0]["generated_text"]
```

```
# =====  
# 13. DISPLAY ANSWER  
# =====
```

```
print("\nAssistant:", answer)
```

```
print("\nProgram completed.")
```

output:



The image shows a screenshot of a code editor and a terminal window. The code editor on the left contains a Python script named `bb.py` located at `C:/Users/Dhanyashree/bb.py`. The script is a document-based AI assistant. It starts by loading document chunks into a context. It then defines a prompt for an LLM, instructing it to answer questions based on the provided context. The script uses the `llm` function from the `openai` library to generate answers. It also includes a fallback message if the answer is not found in the document. The script ends by displaying the generated answer.

```
context += chunks[i] + "\n"

# =====
# 11. CREATE PROMPT
# =====

prompt = """
You are a document-based AI assistant.
Answer the user's question using only the
information in the provided document context.
If the answer is not available, say:
"I could not find the answer in the document."
Document Context:
{}
Question:
{}
Answer:
""".format(
    context,
    question
)

# =====
# 12. GENERATE ANSWER
# =====

result = llm(
    prompt,
    max_new_tokens=100
)

answer = result[0]["generated_text"]

# =====
# 13. DISPLAY ANSWER
# =====

print("\nAssistant:", answer)

print("\nProgram completed.")
```

The terminal window on the right shows the execution of the script. It starts with a restart message, followed by a series of status messages indicating the successful loading of the document, the embedding model, and the language model. It then displays the AI assistant's readiness and the user's questions and answers.

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dhanyashree/bb.py =====
DOCUMENT-BASED AI ASSISTANT
External document loaded successfully.
Number of document chunks: 6
Loading embedding model...
Embedding model loaded.
Creating document embeddings...
Embeddings created successfully.
Creating FAISS vector database...
Vector database created.
Documents stored: 6
Loading AI language model...
Device set to use cpu
Language model loaded.
=====
AI ASSISTANT READY
=====
Ask questions about the external document.
Type 'exit' to stop.
You: What is artificial intelligence?
Assistant: a branch of computer science that focuses on creating intelligent sys
tems
You: What is deep learning?
Assistant: a type of Machine Learning that uses artificial neural networks with
```

Ln 53 Col 0